Stocarea experimentelor cu NumPy și SQLite

https://gemini.google.com/app/ef20d6a65e2b305d

User prompt: Cerinta: Livrabil 2 CONTEXT COMUN – proiect CelLab (Python 3.11, numpy, pydantic v2, pytest) Construim o aplicație de automate celulare, în echipă de 4. Componentele comunică DOAR prin: 1) SPEC-ul automatului (JSON). Exemplu Game of Life: { "name": "Game of Life", "kind": "neighbour_count", "grid": {"rows": 50, "cols": 50, "boundary": "toroidal"}, "states": ["DEAD", "ALIVE"], "neighborhood": "moore", "rules": [{"from": "DEAD", "to": "ALIVE", "if": {"count": "ALIVE", "in": [3]}}, {"from": "ALIVE", "to": "DEAD", "if": {"count": "ALIVE", "not_in": [2, 3]}}], "otherwise": "unchanged" } - kind: deocamdată doar "neighbour_count". - grid.boundary: "toroidal" (marginile se leagă) sau "fixed" (în afara grilei se consideră starea cu codul 0). - states: lista numelor; CODUL unei stări = indexul ei în listă. - neighborhood: "moore" (8 vecini) sau "von_neumann" (4 vecini). - O regulă: {"from": S, "to": T, "if": C (opțional), "probability": p (opțional, implicit 1.0)} C are "count": Z (starea numărată printre vecini) și EXACT UNA dintre forme: {"in": [..]} | {"not_in": [..]} | {"min": a} | {"max": b} | {"min": a, "max": b} Fără "if" = regulă necondiționată. - Semantică: actualizare SIMULTANĂ (toți citesc generația veche). Pentru fiecare celulă se aplică PRIMA regulă (în ordinea din listă) cu "from" egal cu starea celulei și condiția adevărată; dacă există "probability" < 1, regula se aplică doar cu acea probabilitate (rng = numpy Generator primit ca parametru). Dacă nicio regulă nu se aplică, celula rămâne neschimbată. 2) GRILA: np.ndarray 2D, dtype np.int8, shape (rows, cols), valori = coduri de stare. 3) GENERAȚIILE: list[np.ndarray], generations[0] = lumea inițială. Fișiere și semnături FIXE (nu le schimba numele): - src/parser/schema.py: class AutomatonSpec (pydantic), parse_spec(data: dict) -> AutomatonSpec, load_spec(path) -> AutomatonSpec, class SpecError(Exception) cu atributul .errors: list[str], metode AutomatonSpec.state_code(name) -> int și AutomatonSpec.to_dict() -> dict. În model, câmpul "from" se numește from_ (alias "from"), "if" se numește if_ (alias "if"). - src/engine/simulator.py: step(grid, spec, rng=None) -> np.ndarray; run(grid, spec, n, rng=None) -> list[np.ndarray] - src/analysis/questions.py: state_after(grid, spec, n, rng=None); count_cells(grid, spec, state_name) -> int - src/world/initial.py: from_text(text, spec, symbols=None), random_world(spec, densities, seed, rows=None, cols=None), predefined(name, spec, rows=None, cols=None), to_text(grid, spec, symbols=None) - src/storage/experiments.py: ExperimentConfig, save_experiment(...), load_experiment(exp_id), list_experiments() Structura: src/{parser,engine,analysis,world,storage}, specs/, tests/. Fiecare folder din src are __init__.py. [CONTEXT COMUN] Sarcina mea: src/storage/experiments.py și scriptul demo_d2.py. 1. ExperimentConfig (dataclass sau pydantic): generations, seed (opțional), store_intermediate (bool), store_every (int, implicit 1), init_method ("manual" | "random" | "predefined"), init_params (dict). 2. save_experiment(spec, initial_grid, config, generations, base_dir="experiments") -> exp_id: - folder experiments/<exp_id>/ cu spec.json (spec.to_dict()), config.json, generations.npz (np.savez_compressed; dacă store_intermediate e False, salvează doar prima și ultima generație, altfel fiecare a store_every-a generație, plus ultima, mereu; păstrează și indicii generațiilor); - un rând în experiments/index.db (SQLite): id, nume automat, dată, generații, seed, cale. 3. load_experiment(exp_id) -> obiect cu spec (AutomatonSpec), config, generations și indicii lor. list_experiments() -> listă din SQLite, cele mai noi primele. 4. tests/test_storage.py: round-trip (salvez, încarc, grilele sunt identice cu np.array_equal), store_every funcționează, un experiment aleator re-rulat cu același seed dă aceleași generații. Folosește tmp_path. 5. demo_d2.py (pentru prezentare): pentru Game of Life, Brian's Brain și Forest Fire din specs/: încarcă spec-ul, creează lumea inițială (from_text sau predefined), rulează 10 pași, afișează în terminal primele 3 generații cu to_text, răspunde la „câte celule de tip X după N pași", salvează experimentul, îl reîncarcă și afișează „Reîncărcare OK" dacă e identic. Explică-mi de ce .npz + SQLite și cum aș regăsi un

experiment vechi.

Response: Combinația dintre .npz și SQLite oferă un echilibru optim între performanța stocării volumelor mari de date și ușurința căutării metadatelor, îndeplinind eficient cerințele de stocare și recuperare a stărilor pentru Livrabilul 2.

De ce .npz pentru generații și SQLite pentru indexare

- **Eficiența spațiului și a vitezei (.npz):** Simulările de automate celulare produc zeci sau sute de matrici 2D la fiecare rulare. Salvarea acestora în format text sau JSON ar fi lentă și ar ocupa enorm de mult spațiu. Formatul .npz (np.savez_compressed) arhivează binar și comprimă nativ matricile NumPy (np.int8), garantând un I/O rapid.
- **Interogare instantanee (SQLite):** Un fișier index.db îți permite să gestionezi istoricul simulărilor. Poți lista rapid experimentele, sorta după dată sau filtra după numele automatului (ex. "Game of Life") folosind SQL, fără a fi nevoie să parcurgi folderele de pe disc și să deschizi zeci de fișiere config.json.
- **Decuplare arhitecturală:** Metadatele (ușoare și frecvent interogate) rămân în baza de date centralizată, în timp ce stările grilei (grele și accesate doar la nevoie) sunt izolate în fisierele de pe disc.

Cum regăsești un experiment vechi

1. **Apelezi list_experiments():** Această funcție interoghează tabelul din index.db și returnează o listă cu toate rulările anterioare, cele mai noi fiind primele. Aici poți vedea detalii precum numele automatului, data rulării, numărul de generații salvate și seed-ul folosit.
2. **Identifici exp_id-ul:** Din rezultatele listate, alegi identificatorul unic (UUID sau timestamp) al experimentului pe care vrei să-l inspectezi.
3. **Apelezi load_experiment(exp_id):** Sistemul accesează direct folderul experiments/<exp_id>/ și:
    - Reconstruiește obiectul AutomatonSpec citind spec.json.
    - Încarcă parametrii de rulare din config.json (inclusiv instanța de ExperimentConfig).
    - Extrage vectorii 2D din generations.npz sub formă de list[np.ndarray], corelându-i cu indicii corespunzători (în caz că store_every a fost > 1).
4. **Reiei simularea sau analiza:** Obiectul returnat este gata pentru a fi trimis către analysis/questions.py sau pentru a fi afișat în terminal de scriptul demo_d2.py, confirmând că starea recuperată este identică cu cea inițială.

User prompt: De indeplinit sarcina mea: src/storage/experiments.py și scriptul demo_d2.py.

Response: Iată implementarea completă pentru fișierul src/storage/experiments.py și scriptul demo_d2.py, respectând arhitectura impusă, cerințele pentru Livrabilul 2 și tehnologiile alese (Python 3.11, numpy, pydantic, sqlite3).

1. Modulul de stocare: src/storage/experiments.py

Acest modul conține logica pentru salvarea și încărcarea simulărilor, arhiva .npz pentru stări și baza de date SQLite pentru metadate.

```
import json
import sqlite3
import uuid
```

```
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, List, Optional
from pydantic import BaseModel, Field
import numpy as np

# Se presupune existența parserului conform specificațiilor
from src.parser.schema import AutomatonSpec, parse_spec

class ExperimentConfig(BaseModel):
    generations: int
    seed: Optional[int] = None
    store_intermediate: bool = True
    store_every: int = 1
    init_method: str = Field(..., pattern="^(manual|random|predefined)$")
    init_params: Dict[str, Any] = Field(default_factory=dict)

class ExperimentData(BaseModel):
    spec: AutomatonSpec
    config: ExperimentConfig
    generations: List[np.ndarray]
    indices: List[int]

    class Config:
        arbitrary_types_allowed = True

def _get_db_connection(base_dir: Path) -> sqlite3.Connection:
    db_path = base_dir / "index.db"
    conn = sqlite3.connect(db_path)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS experiments (
            id TEXT PRIMARY KEY,
            name TEXT,
            date TEXT,
            generations INTEGER,
            seed INTEGER,
            path TEXT
        )
    """)
    conn.commit()
    return conn

def save_experiment(
    spec: AutomatonSpec,
    initial_grid: np.ndarray,
    config: ExperimentConfig,
    generations: List[np.ndarray],
    base_dir: str = "experiments"
) -> str:
    base_path = Path(base_dir)
    base_path.mkdir(parents=True, exist_ok=True)
```

```
    exp_id = str(uuid.uuid4())
    exp_dir = base_path / exp_id
    exp_dir.mkdir(exist_ok=True)

    # 1. Salvare spec.json
    with open(exp_dir / "spec.json", "w", encoding="utf-8") as f:
        json.dump(spec.to_dict(), f, indent=2)

    # 2. Salvare config.json
    with open(exp_dir / "config.json", "w", encoding="utf-8") as f:
        json.dump(config.model_dump(), f, indent=2)

    # 3. Salvare generations.npz
    indices = []
    arrays_to_save = {}
    total_gens = len(generations)

    for i, grid in enumerate(generations):
        # Determinăm dacă salvăm indexul curent
        save_this = False
        if i == 0 or i == total_gens - 1:
            save_this = True
        elif config.store_intermediate and i % config.store_every == 0:
            save_this = True

        if save_this and i not in indices:
            indices.append(i)
            arrays_to_save[f"gen_{i}"] = grid

    arrays_to_save["indices"] = np.array(indices, dtype=np.int32)
    np.savez_compressed(exp_dir / "generations.npz", **arrays_to_save)

    # 4. Inserare în baza de date SQLite
    conn = _get_db_connection(base_path)
    date_str = datetime.now().isoformat()
    try:
        conn.execute(
            "INSERT INTO experiments (id, name, date, generations, seed, path)
VALUES (?, ?, ?, ?, ?, ?)",
            (exp_id, spec.name, date_str, config.generations, config.seed,
str(exp_dir))
        )
        conn.commit()
    finally:
        conn.close()

    return exp_id

def load_experiment(exp_id: str, base_dir: str = "experiments") ->
ExperimentData:
    exp_dir = Path(base_dir) / exp_id
    if not exp_dir.exists():
```

```
        raise FileNotFoundError(f"Experimentul {exp_id} nu a fost găsit.")

    # Încărcare spec
    with open(exp_dir / "spec.json", "r", encoding="utf-8") as f:
        spec_data = json.load(f)
        spec = parse_spec(spec_data)

    # Încărcare config
    with open(exp_dir / "config.json", "r", encoding="utf-8") as f:
        config_data = json.load(f)
        config = ExperimentConfig(**config_data)

    # Încărcare matrice
    npz_data = np.load(exp_dir / "generations.npz")
    indices = npz_data["indices"].tolist()

    loaded_generations = []
    for idx in indices:
        loaded_generations.append(npz_data[f"gen_{idx}"])

    return ExperimentData(
        spec=spec,
        config=config,
        generations=loaded_generations,
        indices=indices
    )

def list_experiments(base_dir: str = "experiments") -> List[Dict[str, Any]]:
    base_path = Path(base_dir)
    if not (base_path / "index.db").exists():
        return []

    conn = _get_db_connection(base_path)
    try:
        cursor = conn.execute("SELECT id, name, date, generations, seed, path
FROM experiments ORDER BY date DESC")
        columns = [col[0] for col in cursor.description]
        results = [dict(zip(columns, row)) for row in cursor.fetchall()]
        return results
    finally:
        conn.close()
```

2. Scriptul demonstrativ: demo_d2.py

Acest script pune cap la cap parserul, motorul de simulare, modulul de întrebări analitice și stocarea implementată anterior, demonstrând funcționalitatea aplicației pentru prezentare.

```
import os
import numpy as np
from pathlib import Path
```

```
# Importuri din modulele proiectului (asumând structura Cerinței 2)
from src.parser.schema import load_spec
from src.world.initial import predefined, to_text
from src.engine.simulator import run
from src.analysis.questions import count_cells
from src.storage.experiments import ExperimentConfig, save_experiment,
load_experiment

def demo_automaton(spec_filename: str, state_to_count: str, init_name: str =
"glider"):
    print(f"\n{'='*50}")
    print(f" DEMO: {spec_filename}")
    print(f"{'='*50}")

    spec_path = Path("specs") / spec_filename
    if not spec_path.exists():
        print(f"Fișierul {spec_path} lipsește! Creează-l pentru a rula
demo-ul.")
        return

    # 1. Încărcare spec
    spec = load_spec(str(spec_path))
    print(f"Specificație încărcată cu succes: {spec.name}")

    # 2. Lume inițială
    # Pentru simplicitate apelăm "predefined"; fallback pe creare aleatoare
dacă nu e definit
    try:
        initial_grid = predefined(init_name, spec, rows=spec.grid.rows,
cols=spec.grid.cols)
    except Exception:
        # Fallback (în caz că 'predefined' nu are acest pattern implementat)
        initial_grid = np.zeros((spec.grid.rows, spec.grid.cols),
dtype=np.int8)
        initial_grid[1, 1] = spec.state_code(spec.states[1])

    # Configurare experiment
    steps = 10
    config = ExperimentConfig(
        generations=steps,
        store_intermediate=True,
        store_every=1,
        init_method="predefined",
        init_params={"name": init_name}
    )

    # 3. Rulare
    rng = np.random.default_rng(42)  # Seed fix pentru determinism în demo
    generations = run(initial_grid, spec, steps, rng=rng)

    # 4. Afișare primele 3 generații
```

```
    print("\n--- Primele 3 generații ---")
    for i in range(min(3, len(generations))):
        print(f"Generația {i}:")
        print(to_text(generations[i], spec))
        print("-" * 20)

    # 5. Întrebare analitică
    count_after_10 = count_cells(generations[-1], spec, state_to_count)
    print(f"Q: Câte celule '{state_to_count}' există după {steps} pași?")
    print(f"A: {count_after_10}")

    # 6. Salvare experiment
    print("\n--- Salvare experiment ---")
    exp_id = save_experiment(spec, initial_grid, config, generations)
    print(f"Experiment salvat cu ID: {exp_id}")

    # 7. Încărcare și verificare (Round-trip)
    print("\n--- Încărcare experiment ---")
    loaded_data = load_experiment(exp_id)

    is_identical = True
    if len(generations) != len(loaded_data.generations):
        is_identical = False
    else:
        for orig, loaded in zip(generations, loaded_data.generations):
            if not np.array_equal(orig, loaded):
                is_identical = False
                break

    if is_identical:
        print("Reîncărcare OK: Grilele sunt identice!")
    else:
        print("EROARE: Grilele încărcate diferă de cele originale.")

if __name__ == "__main__":
    # Asigurăm folderul specs
    os.makedirs("specs", exist_ok=True)

    # Exemplu de rulări:
    # Va trebui să existe fișierele corespunzătoare în folderul "specs"
    demo_automaton("game_of_life.json", state_to_count="ALIVE",
init_name="glider")
    demo_automaton("brians_brain.json", state_to_count="FIRING",
init_name="oscillator")
    demo_automaton("forest_fire.json", state_to_count="FIRE",
init_name="random_forest")
```
